

```
import pandas as pd
```

```
class Processo:
```

```
    def __init__(self, id, tamanho):
```

```
        self.id = id
```

```
        self.tamanho = tamanho
```

```
        self.tempo_espera = 0
```

```
    def __repr__(self):
```

```
        return f"P{self.id}({self.tamanho}u)"
```

```
class Particao:
```

```
    def __init__(self, id, tamanho):
```

```
        self.id = id
```

```
        self.tamanho = tamanho
```

```
        self.processo_ocupante = None
```

```
    @property
```

```
    def ocupada(self):
```

```
        return self.processo_ocupante is not None
```

```
    @property
```

```
    def fragmentacao_interna(self):
```

```
        if self.ocupada:
```

```
            return self.tamanho - self.processo_ocupante.tamanho
```

```
        return 0
```

```
class GerenciadorMemoria:
```

```
    def __init__(self, tamanhos_particoes):
```

```
        self.particoes = [Particao(i, tam) for i, tam in enumerate(tamanhos_particoes)]
```

```
        self.fila_espera = []
```

```
        self.historico_processos_espera = set()
```

```
        self.total_ciclos = 0
```

```
        self.memoria_total = sum(tamanhos_particoes)
```

```
        self.soma_utilizacao = 0
```

```
        self.soma_frag_interna = 0
```

```
    def alocar(self, processo):
```

```
        self.total_ciclos += 1
```

```
        self._incrementar_espera_fila()
```

```
        alocado = self._tentar_alocacao_direta(processo)
```

```
        if not alocado:
```

```
            print(f"-> [!] Sem espaço para {processo}. Indo para fila de espera.")
```

```
            self.fila_espera.append(processo)
```

```
            self.historico_processos_espera.add(processo.id)
```

```

        self._registrar_metricas()

def liberar(self, id_processo):
    self.total_ciclos += 1
    self._incrementar_espera_fila()

    for p in self.particoes:
        if p.ocupada and p.processo_ocupante.id == id_processo:
            print(f"<- [V] Liberando partição {p.id} (ocupada por {p.processo_ocupante})")
            p.processo_ocupante = None
            self._verificar_fila()
            break

    self._registrar_metricas()

def _tentar_alocacao_direta(self, processo):
    for particao in self.particoes:
        if not particao.ocupada and particao.tamanho >= processo.tamanho:
            particao.processo_ocupante = processo
            print(f"-> [OK] {processo} alocado na Partição {particao.id} ({particao.tamanho}u)")
            return True
    return False

def _verificar_fila(self):
    """Swapping: Tenta trazer processos da fila para a memória"""
    for i, processo in enumerate(self.fila_espera):
        if self._tentar_alocacao_direta(processo):
            self.fila_espera.pop(i)
            break

def _incrementar_espera_fila(self):
    for p in self.fila_espera:
        p.tempo_espera += 1

def _registrar_metricas(self):
    espaco_ocupado = sum(p.processo_ocupante.tamanho for p in self.particoes if
p.ocupada)
    frag_total = sum(p.fragmentacao_interna for p in self.particoes)

    self.soma_utilizacao += (espaco_ocupado / self.memoria_total)
    self.soma_frag_interna += (frag_total / self.memoria_total)

def relatorio_final(self):
    total_espera = sum(p.tempo_espera for p in self.fila_espera) + \
        sum(p.tempo_espera for p in self.historico_processos_espera if isinstance(p,
Processo))

```

```

        media_espera = total_espera / len(self.historico_processos_espera) if
self.historico_processos_espera else 0

    print("\n" + "="*40)
    print("    RESULTADOS DA SIMULAÇÃO")
    print("="*40)
    print(f"Processos que esperaram:    {len(self.historico_processos_espera)}")
    print(f"Tempo médio de espera:    {media_espera:.2f} ciclos")
    print(f"Utilização média:          {(self.soma_utilizacao/self.total_ciclos)*100:.2f}%")
    print(f"Fragmentação Interna Total:
{(self.soma_frag_interna/self.total_ciclos)*100:.2f}%")
    print("="*40)

```

```

gm = GerenciadorMemoria([100, 200, 300, 400])

```

```

processos = [
    Processo(1, 50),
    Processo(2, 250),
    Processo(3, 350),
    Processo(4, 150),
    Processo(5, 450),
    Processo(6, 90),
]

```

```

for p in processos:
    gm.alocar(p)

```

```

gm.liberar(2)
gm.liberar(3)

```

```

gm.relatorio_final()

```